Deployment

> Part of the Python API Backend curriculum. This document covers running a FastAPI application in production: process management, containers, cloud platforms, reverse proxies (including ordered path rules and edge policies), data refreshes and process-level caches, CI/CD, and health checks.

1. From development to production

In development, `uvicorn app.main:app --reload` is enough — one process, auto-restarting on code changes, no concern for concurrent load. Production has different requirements: handling concurrent requests reliably, restarting automatically after a crash, running behind a layer that handles HTTPS and routing, and being deployed repeatably rather than by hand.

2. Running uvicorn in production

2.1 Multiple worker processes

A single Python process can only use one CPU core effectively for synchronous work (see Fundamentals, §4.5 for the async/sync distinction) — running multiple **worker processes** lets a service use multiple cores and continue serving requests if one worker crashes:

```
uvicorn app.main:app --workers 4 --host 0.0.0.0 --port 8000
```

`--reload` and `--workers` are mutually exclusive — reload is a development-only convenience. A common production alternative is **Gunicorn** managing multiple **uvicorn worker processes** (Gunicorn handles process supervision — restarting a crashed worker automatically — while uvicorn's workers handle the actual ASGI serving):

```
gunicorn app.main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Remember from API Design, §1.1: each worker process has its own independent in-memory cache — if you need a cache shared across workers, use an external store like Redis instead of relying on `lru_cache`.

3. Containers

Docker packages an application together with everything it needs to run (Python version, dependencies, code) into a single, portable image:

```dockerfile
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t my-api .
docker run -p 8000:8000 my-api
```

Containerizing gives you the same "build once, run anywhere" guarantee discussed in the architecture overview's configuration principle — the exact same image runs identically on a developer's laptop, a staging server, and production, with environment-specific behavior driven entirely by environment variables passed in at run time (`docker run -e DATABASE_URL=... my-api`), never baked into the image.

4. Reverse proxies

A **reverse proxy** (nginx, Caddy, or a cloud load balancer) typically sits in front of the application server and handles:

- **HTTPS/TLS termination** — encrypting traffic to the client, so the application itself only needs to speak plain HTTP internally.
- **Routing multiple services under one domain** — e.g., serving a frontend's static files at `/` and proxying `/api/*` requests through to the backend API process.
- **Deploy-path prefix handling** — if the whole application is served under a sub-path (`example.com/my-app/`) behind the proxy, the proxy (or middleware in the application itself, see Fundamentals, §4.7) needs to consistently strip or forward that prefix so the application's own routes (mounted at plain paths) still resolve correctly.

Path rules are ordered, and ordering is load-bearing. Gateways and tunnels that match paths top-down will happily let an *unanchored* prefix rule (`/app`) swallow a more specific route (`/app/api`) listed below it, answering with the frontend's `index.html` and a 200. Put specific rules first, anchor them (`^/app/api`), and verify each with a real request. Also check whether the gateway forwards the full prefixed path or strips it — an application that assumes the opposite 404s on every route.

Edge rules do real work. A gateway in front of a public API is the natural home for rate limiting, security headers and — for operational routes — identity-provider login or service-token policies, all enforced before a request reaches your process. Prefer them to re-implementing the same checks in application code, and keep a short inventory of which path patterns each rule matches so a new route does not accidentally fall outside it.

A minimal nginx config fragment illustrating the routing concern:

```
location /api/ {
    proxy_pass http://localhost:8000/;
    proxy_set_header Host $host;
}
```

4.1 Data refreshes and process-level caches

An API that caches its loaded data in-process (`@lru_cache`) only sees a new file after the process restarts. When a scheduled job refreshes the underlying data, the job — not a human — must restart (or invalidate) the API, **after** the new data has validated, and a failed restart should be reported but not treated as a failure of the refresh itself. The refresh job should also: back up the previous download, validate the new one against it (hard-fail thresholds that restore the backup; soft-flag thresholds that raise an alert but publish), run each pipeline stage as its own failure domain, and send one summary notification per run carrying the highest severity. See Data Science, Multi-Source Pipelines, §9.

5. Cloud platform options

Option	What it is	Good fit for
Container platforms (AWS ECS/Fargate, Google Cloud Run, Azure Container Apps)	Run a container image without managing the underlying servers yourself	Most APIs of this shape — a straightforward middle ground between "manage your own VM" and "fully serverless"
Serverless functions (AWS Lambda via an adapter like Mangum, Google Cloud Functions)	The platform runs your code only in response to a request, scaling to zero when idle	Spiky, infrequent traffic where paying only per-invocation matters more than avoiding cold-start latency
A plain virtual machine (EC2, a VPS)	You manage the OS, process supervision, and everything else yourself	Maximum control, at the cost of more of everything above being your own responsibility

For a small analytics API with predictable, moderate traffic, a managed container platform (Cloud Run, ECS/Fargate) is usually the best balance of "someone else manages the infrastructure" and "runs continuously with predictable latency" without the added architectural complexity of adapting a normal web framework to a serverless request/response model.

6. CI/CD

A typical continuous integration/continuous deployment pipeline for an API:

1. **On every push**: run the test suite (Testing) and a linter/type checker.
2. **On merge to main** (or a release trigger): build the container image, tag it, push it to a registry.
3. **Deploy**: trigger the hosting platform to roll out the new image — most managed platforms support this with a single command or a webhook from your CI system.

The key property to preserve: **nothing should be deployed that hasn't passed the automated test suite** — the whole point of the testing discipline in Testing is undermined if a broken build can still reach production because a human forgot to run the tests locally first.

7. Health checks

Most hosting platforms and orchestrators expect a **health check** endpoint — a cheap, side-effect-free route the platform polls periodically to decide whether an instance is healthy enough to keep receiving traffic:

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

Keep this endpoint intentionally minimal — it should reflect "is this process alive and able to respond at all," not "is every downstream data dependency currently available" (that's what the per-endpoint `503` handling from API Design, §2 is for). Conflating the two would cause the platform to kill and restart otherwise-healthy instances just because an optional upstream data file hasn't been generated yet.

8. Core tools summary

Tool	Used for
uvicorn / **Gunicorn**	running the ASGI application, with multiple worker processes
Docker	packaging the application and its dependencies into a portable, reproducible image
nginx (or a cloud load balancer)	reverse proxying, HTTPS termination, routing
A CI system (GitHub Actions and similar)	running tests and building/deploying images automatically

See also

- Python API Backend index
- Testing — the test suite a CI pipeline should run before any deployment
- Architecture Overview, §11 for how this fits alongside a frontend's own deployment